

Below znajdziesz kompletny, „end-to-end” plan implementacji – od repozytorium po przepływy sieciowe – dla pakietu routów, o które prosisz.
Staralem się tak podzielić całość, abyś mógł przekleić poszczególne sekcje jako gotowe issue/epiki do Jira / GitHub Projects.

1. Stos technologiczny

Warstwa	Tech	Kluczowe paczki npm
---------	------	---------------------

Frontend (app web)	Next.js 14 (app router), React 18, TypeScript	next, react, @tanstack/react-query, zustand, tailwindcss, clsx, framer-motion, wagmi, viem, @rainbow-me/rainbowkit, next-auth@beta (provider Google + custom SIWE)
Backend (app api)	NestJS, Node 20, TypeScript	@nestjs/*, passport, passport-google-oauth20, viem, ethers, jsonwebtoken, class-validator, zod, prisma, @prisma/client, axios, bullmq, socket.io, @circle-fin/* (lub własne thin SDK nad REST)
Baza danych	PostgreSQL 15 (+ Prisma ORM)	pg, @planetscale/database*
Cache / kolejki	Redis 7 (BullMQ)	bullmq, ioredis
Dev-&-Ops	Turborepo (workspaces), pnpm, Docker, GitHub Actions, Sentry, Datadog	@turbo, pnpm, dotenv, @sentry/*

* Planetscale opcjonalnie – na wypadek, gdybyś potrzebował MySQL.

2. Struktura monorepo (tipjar/)

```
tipjar/
├── apps/
│   ├── web/      # Next.js
│   └── api/      # NestJS
├── packages/
│   ├── ui/       # Design-system React + Tailwind
│   ├── db/       # Prisma schema + generated client
│   ├── circle-sdk/ # Thin wrapper na REST Circle (Wallets, Payments, Payouts)
│   └── eslint-config/
├── .github/
│   └── workflows/ci.yml
├── docker-compose.yml # postgres, redis, localstack (webhooks)
└── turbo.json
```

Monorepo przyspiesza współ-dzielenie typów (db package) i pojedyncze CI.

3. Konfiguracja Circle (sekrety w api/.env)

```
CIRCLE_API_KEY=prod_xxx
CIRCLE_WALLET_SET_ID=ws_...
CIRCLE_GAS_STATION_API_KEY=gs_...
CIRCLE_PAYMASTER_ADDRESS=0x3B...
CIRCLE_PAYOUT_PROFILE_ID=pp_...
# On-ramp
CIRCLE_CARD_MASTER_WALLET_ID=wl_...
CIRCLE_WEBHOOK_SECRET=whsec_...
```

> Tip ops: własne środowisko Sandbox spinamy równolegle; w CI używamy flag
CIRCLE_ENV=sandbox.

4. Kluczowe modele (Prisma schema.prisma)

```
model Creator {
  id          String  @id @default(uuid())
  email       String  @unique
  googleId    String? @unique
  ethAddress  String? @unique
  circleWalletId String @unique
  kycStatus   CreatorKycStatus @default(PENDING)
  tips        Tip[]
  payouts     Payout[]
  createdAt   DateTime @default(now())
}

model Tip {
  id          String  @id @default(uuid())
  creatorId   String
  fanId       String?
  amountUSDC  Decimal @db.Decimal(38,6)
  circleTxId  String? // jeśli transfer on-chain
  source      TipSource
  createdAt   DateTime @default(now())
  creator     Creator @relation(fields: [creatorId], references: [id])
}

enum TipSource {
  INTERNAL_WALLET // DCW + Gas Station
```

```
EOA_PAYMASTER // zewn. MetaMask + Paymaster
CARD_ONRAMP    // fiat→USDC
}
```

```
model Payout { ... }
```

5. Szczegółowe przepływy (sequence steps)

5.1 Rejestracja / logowanie twórcy

Google OAuth 2 & SIWE

1. Frontend (/signup)

wybór przycisku Google lub Web3.

2. next-auth → /api/auth/google (PassportStrategy) – uzyskujemy profile i accessToken → walidacja i JWT dla sesji. iwe/nonce → podpis → /auth/siwe/verify → Nest validate & JWT. p** (AuthService.afterLogin`)

circle.wallets.createDeveloperControlledWallet() → zapis circleWalletId w DB. cy (/creator/dashboard)

Backend

GET /creator/stats

→ db.tip.aggregate({ sum, count, groupBy day })

Frontend

React Query useCreatorStats(); wykresy (recharts) + animacja number-roll. /creator/payouts)

1. GET /creator/payout-methods – listujemy bank-accounts zwrócone z /v1/payouts/banks.

2. POST /creator/payout

Walidacja KYC statusu.

```
circle.payouts.createTransfer({ sourceWalletId, destinationId, amount })
```

Zapis w Payout + BullMQ job do pollingu statusu.

5.4 Publiczny profil twórcy ([username])

Komponent TipWidget

Tryb A (fan posiada konto TipJar):

POST /tips/internal-transfer → backend circle.transfers (wallet→wallet) + Gas Station sponsoruje fee . EOA):

Front buduje UserOperation w oparciu o Circle Paymaster v0.8 + podpisany Permit na USDC.

Wysyła do Bundlera → po txHash powrót do Front → webhook UserOperationExecuted zapisuje Tip.

1. POST /tips/card-intent → backend circle.payments.createPayment() (on-ramp).

2. Front przekierowuje do hosted checkout.

3. Webhook payment.successful → backend circle.transfers wewn. na wallet twórcy + zapis Tip. trix (backend Nest)

Method	Path	Guard / middleware	Opis
GET	/auth/siwe/nonce	none	generuje nonce
POST	/auth/siwe/verify		weryfikuje podpis SIWE
GET	/auth/google	Passport-Google	redirect
GET	/creator/stats	JwtAuth + Role(CREATOR)	zsumowane napiwki
POST	/creator/payout	JwtAuth + Role(CREATOR)	tworzy payout w Circle
GET	/creator/payout-methods	JwtAuth + Role(CREATOR)	list bank/card
POST	/tips/internal-transfer	JwtAuth + Role(FAN)	DCW → DCW transfer
POST	/tips/card-intent	optional auth	tworzy PaymentIntent w Circle
POST	/webhooks/circle	Signature Header verify	przyjmuje Payments/Transfers/Payouts

7. Frontend – foldery apps/web

```
pages/  
  [...username]/index.tsx    # public profile  
  creator/  
    dashboard.tsx  
    payouts.tsx  
  _middleware.ts             # next-auth session  
lib/  
  api.ts                     # Axios wrapper  
  circlePaymaster.ts         # Budowanie UserOperation  
stores/  
  useTipStore.ts             # Zustand  
components/  
  TipWidget/  
charts/
```

8. Middleware & observability

Rate-limit 👉 nestjs-rate-limiter na ścieżkach tipowania.

Sentry – @sentry/nextjs & @sentry/nestjs z tracingiem webhooks.

BullMQ – kolejki payout-status-poll oraz payment-settlement.

9. Road-map sprintowa (T-shirt sizing)

Sprint Zakres Output

0 (1 tydz.)	Skeleton monorepo, CI/CD, docker-compose	build green
1 (2 tyg.)	Google OAuth + DB models + Circle wallet provisioning	rejestracja twórcy
end-to-end		
2	SIWE + Gas Station transfer (internal tips)	Fan → Creator (DCW) działający
3	Public profile + TipWidget UI (React) demo	tip \$1 USDC
4	Paymaster flow (EOA) – testnet Arbitrum Sepolia	gasless tip
5	Card on-ramp (hosted checkout) + webhooks	fiat→USDC tip
6	Dashboard stats + payouts MVP	creator cash-out

7 Harden (DDos, rate-limit, Sentry) + E2E tests (Playwright) rc-1

Gotowe – masz pełny „blue-print”, zgodny z dokumentacją UI/UX i technicznymi poradnikami Circle.

Gdybyś chciał rozwinąć któryś z modułów (np. dokładny kod Paymastera albo migracje Prisma) – daj znać!